Group 1

To-Do List Application (Dot)

Anila Narula, Bryan Fuentes-Cortez, Dillon Odum, Ryan Jones, Willie Lin, Zubiyaa Khan

Objective

- The objective of this project is to design and develop a to-do list application that helps users organize, manage, and track their daily tasks efficiently.
- The system aims to provide a simple and user friendly interface where users can create tasks, categorize them, mark them as complete, and search or filter tasks based on their needs.
- This application is intended to improve productivity by allowing users to keep track of their responsibilities in one centralized system while ensuring ease of use, reliability, and quick access to task information.

Method Used – Function Point (FP):

- Inputs → creating and editing tasks
- Outputs → displaying task lists
- Inquiries → searching or viewing tasks
- Internal files → task database

Assign weights based on complexity:

- Simple features = low weight, more complex features = higher weight
- Function Points → **effort (by industry average), cost (based on hourly rate)**

Effort and Cost Estimation:

- Assume ~150 - 200 Function Points, Industry average ~10 hours per FP
- Average developer rate = $50/hour, **Total dev. cost ~$75,000 - $100,000**

Pricing Estimation:

- We add dev. cost, overheads (tools, infrastructure), and profit margin (~20 - 30%)
- Final estimated project price: **~$100,000 - $130,000**

Estimation

Hardware Cost:

- Cloud server hosting (AWS or similar): ~$50 - $100 / month, database hosting: ~$20 - $50 / month. **Annual hardware cost: ~$1,000 - $2,000**

Software Cost:

- Most tools → free (Programming languages, GitHub, Frameworks)
  Optional: IDE licenses- ~$100 - $200 per developer. **Annual cost: ~$500 - $1,500**

Personnel Cost:

- Assumption: 3 - 5 developers, 1 project manager, 1 QA tester
  Estimated salaries: Developers ~ $50 / hour, PM ~ $60 / hour, QA ~ $40 / hour.
  Total personnel cost: ~ $70,000 - $90,000

Timeline

- Project Start Date January 6, 2025 (Monday)
- Project End Date April 11, 2025 (Friday) 14 working weeks
- Total Calendar Duration ~96 calendar days (approx. 3 months)
- Weekends Counted? No weekends Saturday and Sunday are excluded from the working schedule
- Working Days per Week 5 days (Monday through Friday)
- Working Hours per Day 6 hours per developer per day
- Team Size 5 developers
- Total Working Days 70 working days (14 weeks × 5 days)
- Total Effort (Person-Hours) 2,100 person-hours (5 devs × 70 days × 6 hrs)
- Testing(W1-W2), Design(W3-5), Implementing(W5-W9), Testing(W9-W12), Deploy(W13-W14)

Functional

- The system shall allow the user to create a new task by entering a title, description, priority, and due date.
- The system shall allow the user to edit an existing task.
- The system shall allow the user to delete a task.
- The system shall allow the user to mark an existing task as completed.
- The system shall allow the user to view all tasks.
- The system shall allow the user to filter tasks by status (e.g., completed or pending).
- The system shall allow the user to sort tasks by due date.
- The system shall allow the user to sort tasks by priority.
- The system shall allow the user to search for tasks using keywords.
- The system shall allow the user to create a task category.
- The system shall allow the user to edit a task category.
- The system shall allow the user to delete a task category.
- The system shall allow the user to assign tasks to categories.

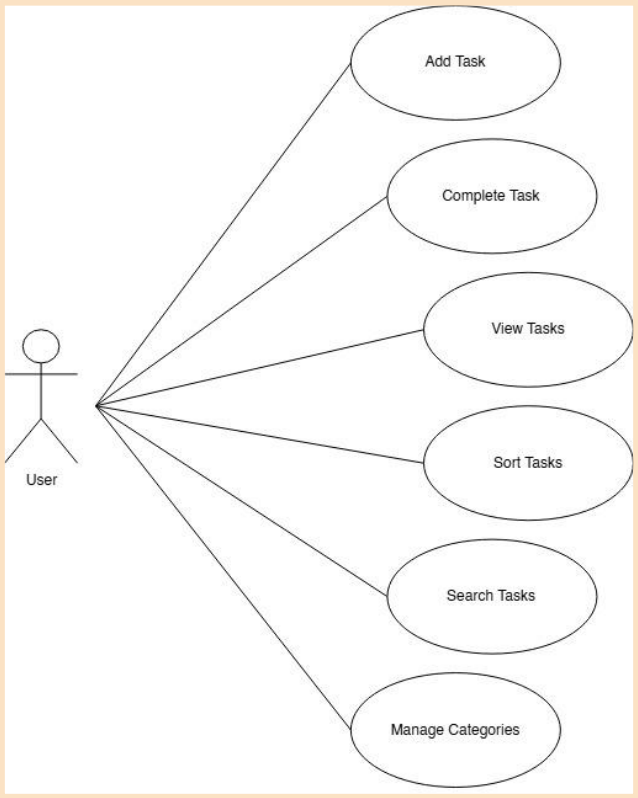
Non-Functional

- **Usability requirement:** The system shall allow a new user to understand and perform basic task operations (add, edit, delete, and complete tasks) within 10 minutes of initial use without formal training.
- **Performance requirement:** The system shall display the task list within 2 seconds of a user request.
- **Space requirement:** The system shall require no more than 100 MB of disk space for application installation and local storage of task data.
- **Efficiency requirement:** The system shall allow task operations such as sorting, filtering, and searching tasks to complete within 2 seconds when managing up to 100 stored tasks.
- **Dependability requirement:** The system shall correctly save all tasks without data loss during normal shutdown.
- **Security requirement:** The system shall ensure that task data stored locally cannot be modified or accessed by unauthorized applications during normal execution.
- **Environmental requirement:** The system shall operate on standard desktop operating systems including Windows, macOS, and Linux with Java installed.

More Non-Functional

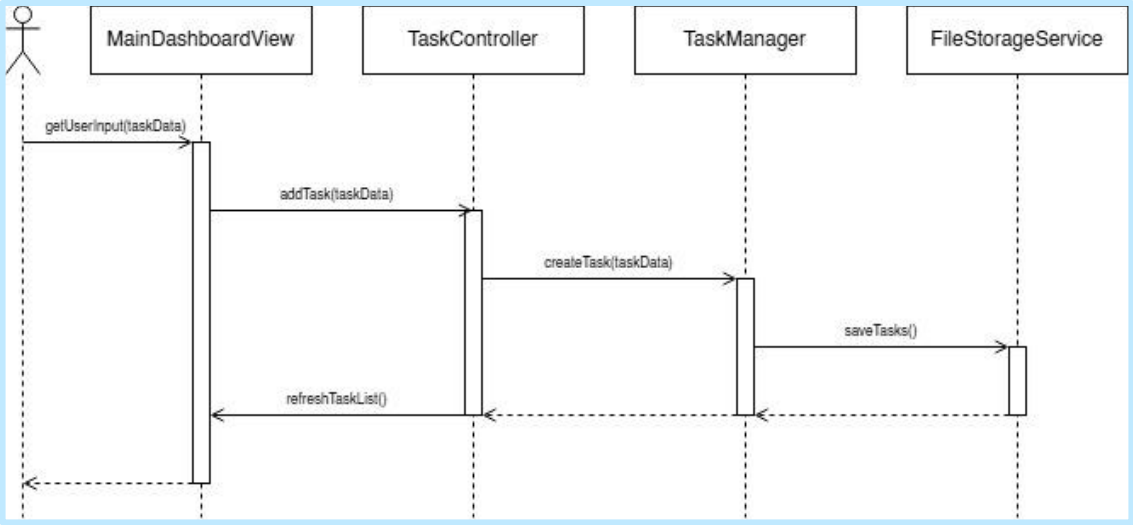
- **Operational requirement:** The system shall support basic mouse and keyboard interactions for all user interface operations.
- **Development requirement:** The system shall be developed using object-oriented programming principles.
- **Regulatory requirement:** The system shall follow the software engineering guidelines and requirements specified in the CS 3354 course project specifications.
- **Ethical requirement:** The system shall not collect or store personally identifiable information beyond user-entered task descriptions.
- **Legislative requirement:** The system shall store all task data locally and shall not transmit user data to external systems.
- **Accounting requirement:** The system shall maintain accurate records of task completion status for tracking completed and pending tasks.
- **Safety/security requirement:** The system shall prevent loss or corruption of task data during unexpected application termination or system crashes.

Use Case


Add Task
Complete Task
View Tasks
Sort Tasks
Search Tasks
Manage Categories
User

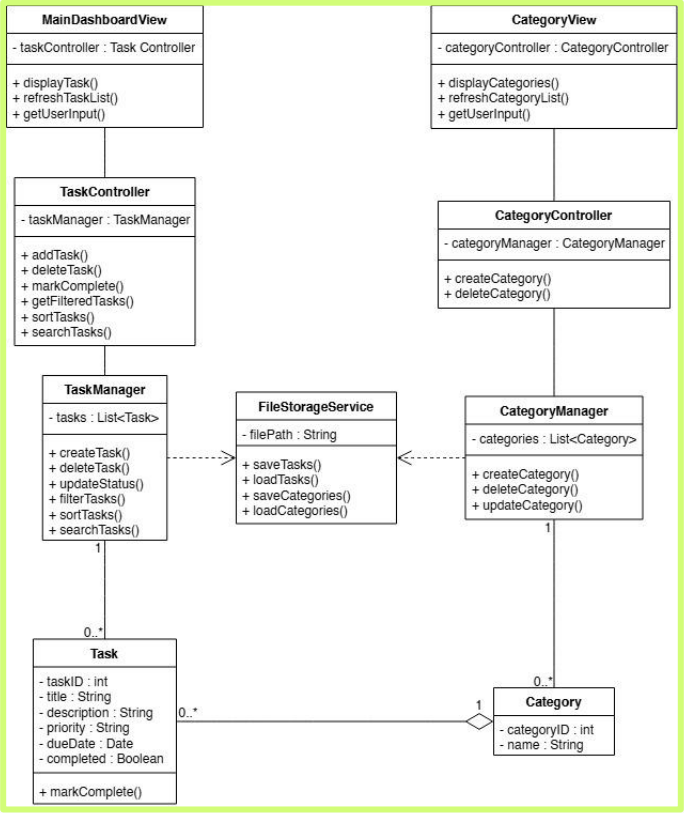
Sequence

Add Task



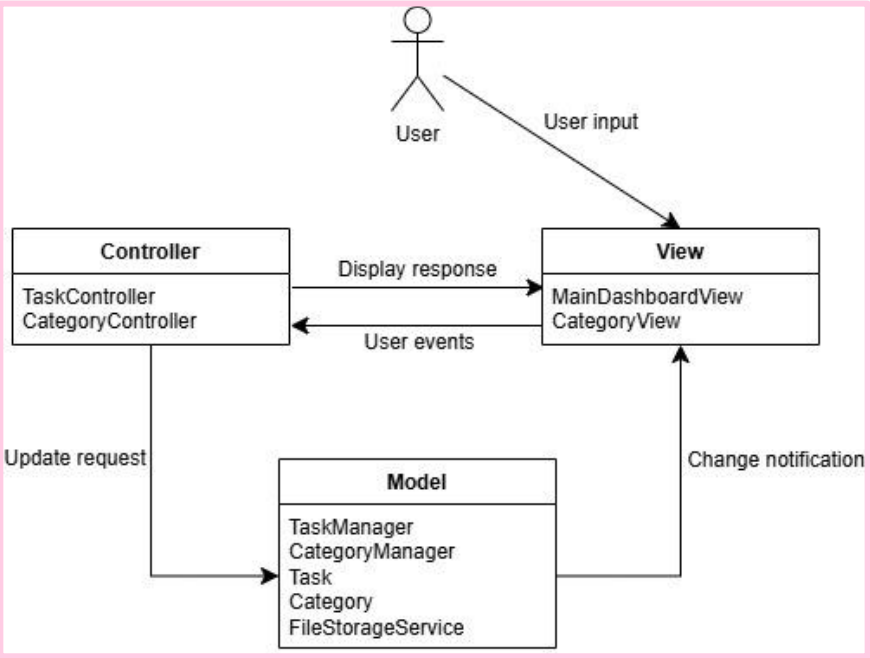
Diagram

Class



Architectural Design

Model-View-Controller (MVC)



Comparison To Similar Designs

Google Tasks

- Personal use
- Requires a Google account
- Integrated with other Google apps like Gmail

Both:

- Simple user interface
- Focus on individual productivity
- Mark tasks as completed

Todoist

- Personal use + small teams
- Task view options: List, Kanban board, calendar
- AI integration: paid

Both:

- Create, edit, delete tasks
- Filter and sort tasks

Microsoft To Do

- Personal use + small teams
- Subtasks
- File attachments
- Cross device sync

Both:

- Emphasize ease of use
- Focus on improving productivity
- Support basic task operations

Zoho Projects

- Team based Project management
- Multi user collaboration
- Time tracking
- Generaton of analytical reports

Both:

- Task creation and management
- Track progress and completion
- Assign deadlines and allow prioritization

Test Plan

This test plan is used to verify the correctness of at least one unit of the To-Do Application. The selected unit for testing is the Task class, which represents an individual task in the application. This unit was selected because task creation and task completion tracking are two core functions of the software.

The test plan focuses on verifying the following behaviors of the Task class:

- a task can be created with a valid name
- an empty task name is rejected
- a task can be marked as completed
- a newly created task defaults to an incomplete state

Testing was performed using automated unit testing in C++ with the GoogleTest framework. Each test case compares the expected behavior of the class against the actual result produced during execution.

A test case is considered successful if the actual result matches the expected result.

Demo Code

Example Task class (C++)

```cpp
#include <string>
#include <stdexcept>
#include <gtest/gtest.h>

//fictitious class for a todo app that manages the task entity itself
//made to demo testing capabilities

class Task {
public:
  Task(const std::string& name)
  {
    if (name.empty()) {
      throw std::invalid_argument("Task name cannot be empty");
    }
    else {
      taskName = name;
    }
    isCompleted = false;
  }

  void markCompleted()
  {
    isCompleted = true;
  }

  bool checkTaskCompletion()
  {
    return isCompleted;
  }

  std::string getName()
  {
    return taskName;
  }

private:
  std::string taskName;
  bool isCompleted = false;
};
```

Test functions to be run against Task class

```cpp
//tests using gtest
TEST(TaskTest, ConstructorStoresName)
{
  Task test("Make Dinner");
  EXPECT_EQ(test.getName(), "Make Dinner");
  EXPECT_FALSE(test.checkTaskCompletion());
}

TEST(TaskTest, DoesNotAcceptEmptyName)
{
  EXPECT_THROW(Task(""), std::invalid_argument);
}

TEST(TaskTest, MarkCompleted)
{
  Task test("Make Lunch");
  test.markCompleted();
  EXPECT_TRUE(test.checkTaskCompletion());
}

TEST(TaskTest, TasksDefaultToNotCompleted)
{
  Task test("Make Lunch");
  EXPECT_FALSE(test.checkTaskCompletion());
}
```

Test suite ran against Task class (GoogleTest)

```
ryan:~/temp > ./run
Running main() from /usr/src/debug/gtest/googletest-1.17.0/go
[==========] Running 4 tests from 1 test suite.
[----------] Global test environment set-up.
[----------] 4 tests from TaskTest
[ RUN      ] TaskTest.ConstructorStoresName
[       OK ] TaskTest.ConstructorStoresName (0 ms)
[ RUN      ] TaskTest.DoesNotExceptEmptyName
[       OK ] TaskTest.DoesNotExceptEmptyName (0 ms)
[ RUN      ] TaskTest.MarkCompleted
[       OK ] TaskTest.MarkCompleted (0 ms)
[ RUN      ] TaskTest.TasksDefaultToNotCompleted
[       OK ] TaskTest.TasksDefaultToNotCompleted (0 ms)
[----------] 4 tests from TaskTest (0 ms total)

[----------] Global test environment tear-down
[==========] 4 tests from 1 test suite ran. (0 ms total)
[  PASSED  ] 4 tests.
```

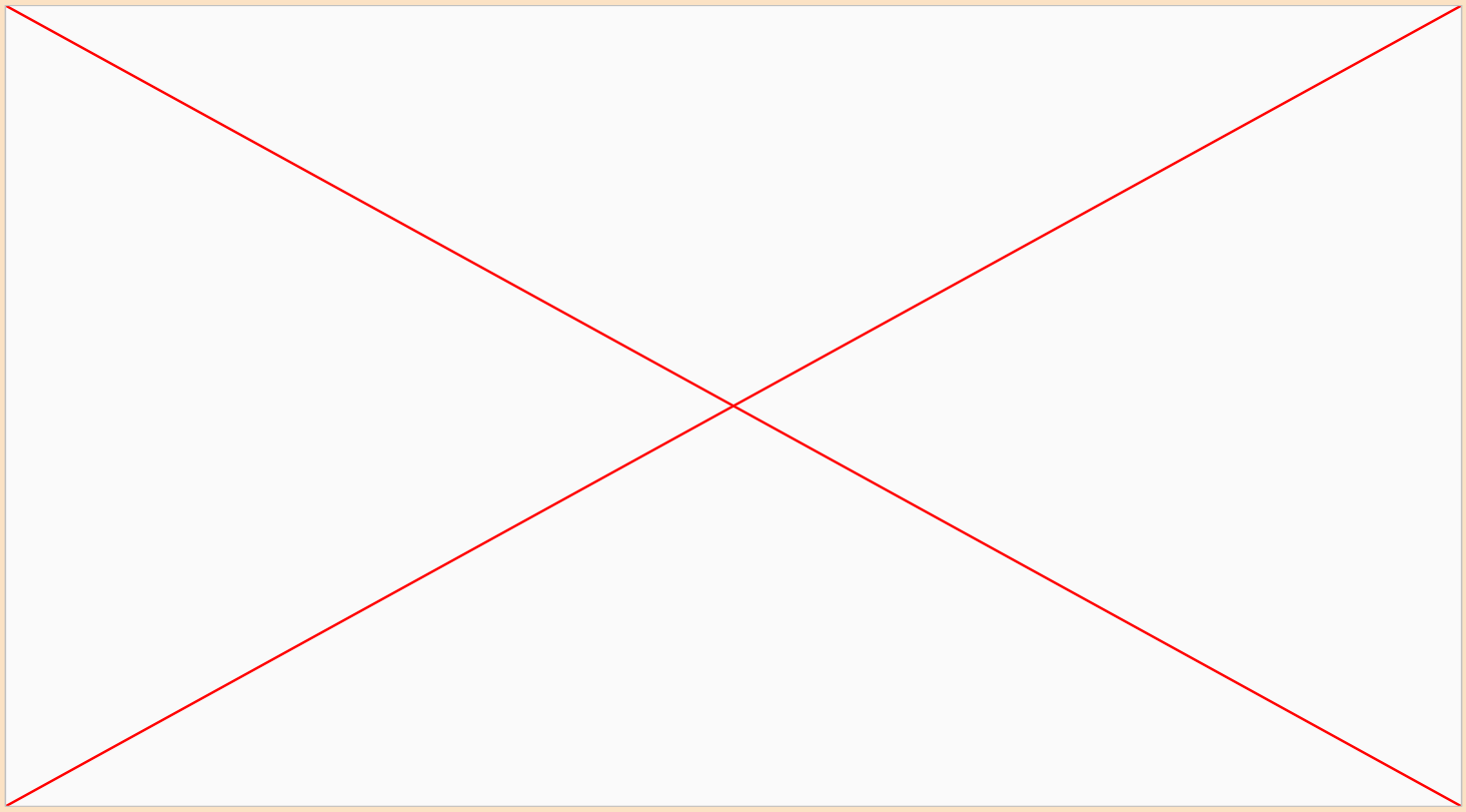
Test Cases

Test	Description	Expected	Actual	Status
TC-1	Create valid name task	Task is created with valid name and starts as incomplete	Task was created with valid name and starts as incomplete	*PASS*
TC-2	Create empty name task	Task creation should throw “invalid_argument” exception and reject Task	Exception was thrown	*PASS*
TC-3	Mark completed Task	Function checkTaskCompleted returns true once Task has been set as complete	Function returns true as expected	*PASS*
TC-4	Check default completion status upon Task creation	A new Task should default to incomplete	Returns false as expected	*PASS*

Test Plan Results

- All four automated test cases passed successfully.
- The results show that the selected unit of the software behaves as intended. The Task class correctly accepts valid input, rejects invalid empty input, updates task completion status when requested, and initializes new tasks in an incomplete state.
- This confirms that the tested unit is functioning correctly. This process would then be expanded upon and repeated for all classes and functions within the project

Prototype



Challenges

Cost Estimation

- FP assumptions hard to pin down
- Dev rates and team size are estimates

Feature Scope

- Core vs. nice features to have trade-off
- Keeping scope realistic

Cross Language consistency

- Java architecture, C++ unit tests
- Keeping diagrams and code aligned

Team Coordination

- Splitting work across 6 members
- Maintaining consistent design

Lessons Learned

1. Clear requirements save time later

Strong specs made UML diagrams and test planning much easier

2. Tests reveal hidden assumptions

GoogleTest forced us to define expected behavior explicitly

3. MVC maps well to team structure

Separate concerns = separate ownership, fewer conflicts

4. Competitor research grounds decisions

Studying Google Tasks and Todoist helped define Dot's niche

Conclusion

13+ Functional requirements

4/4 Unit tests passing

About 115K Estimated cost

- Completed full SE lifecycle: requirements → design → testing → prototype
- C++ chosen over Java for unit tests because of team familiarity with GoogleTest
- MVC architecture refined to cleanly separate FileStorageService

Future directions:

1. Cloud sync
2. User accounts
3. Mobile app
4. Team collaboration
5. Recurring tasks

Thank you!